Problem Statement: Suppose that you are given an m × n grid of integers where each row and each column are in sorted order. Design an O(m + n)-time algorithm that, given as input a sorted m × n grid and an integer, returns whether or not that integer is contained in the grid. Your algorithm should use only O(1) additional space.

Time Complexity: O(m + n)

Program:

```
import java.util.Scanner;
public class SearchMatrix {
static int search(int[][] arr,int m,int n,int key){
int i=0;
int j=n-1;
while(i<m && j>=0){
if(arr[i][j]==key) return 1;
else if(arr[i][j]>key) j--;
else i++;
}}return 0;
}public static void main(String[] args){
Scanner sc=new Scanner(System.in);
int m,n,key;
int[][] arr=new int[100][100];
System.out.print("Enter rows and columns: ");
m=sc.nextInt();
n=sc.nextInt();
System.out.println("Enter sorted matrix:");
for(int i=0;i<m;i++)
for(int j=0;j<n;j++)
arr[i][j]=sc.nextInt();
System.out.print("Enter element to search: ");
key=sc.nextInt();
if(search(arr,m,n,key)==1) System.out.println("Element found");
else System.out.println("Element not found");
sc.close();}}
```

Input / Output:

```
Enter rows and columns: 3 3
Enter sorted matrix:
1 2 5
6 7 8
10 11 12
Enter element to search: 7
Element found
```

Problem Statement: Implement Binary Search and analyse its time complexity on successful and unsuccessful search.

Time Complexity: O(logn)

Program:

```
import java.util.Scanner;
public class BinarySearch {
 static int binSearch(int[] a, int n, int x) {
 int low = 0, high = n - 1;
while (low <= high){
int mid = (low + high) / 2;
if (x < a[mid])
 high = mid - 1;
   else if (x > a[mid])
 low = mid + 1;
    else
 return mid; }
  return -1; }
public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] a = new int[n];
        System.out.println("Enter sorted elements:");
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();
        System.out.print("Enter element to search: ");
        int x = sc.nextInt();
        int result = binSearch(a, n, x);
        if (result != -1)
            System.out.println("Element found at index: " + result);
        else
            System.out.println("Element not found");
        sc.close();}}
```

Input / Output:

```
Enter number of elements: 100
Enter sorted elements:
3 7 12 18 21 25 29 34 38 42 47 51 56 60 64 69 73 78 82 87 91 95 102 108 :
Enter element to search: 219
Element found at index: 43
```

Problem Statement: Sort a given set of elements using the Quicksort method and determine the time required to sort the elements. Repeat the experiment for different values of n and different methods of pivot selection to analyse the behaviour of quick sort. Plot a graph of the time taken versus n. The elements can be read from a file and can be generated using the random number generator.

Time Complexity: O(nlogn)

Program:

```
import java.util.*;
public class quicksort {
static int partition(int[] a, int p, int q) {
  int pivot = a[q];
  int i = p - 1;
for (int j = p; j < q; j++) {
  if (a[j] <= pivot) {
  i++;
  int temp = a[i];
  a[i] = a[j];
  a[j] = temp;
  }}
  int temp = a[i + 1];
  a[i + 1] = a[q];
  a[q] = temp;
 return i + 1;}
static void quickSort(int[] a, int n) {
 Stack<Integer> stack = new Stack<>();
 stack.push(0);
stack.push(n - 1);
while (!stack.isEmpty()) {
 int q = stack.pop();
 int p = stack.pop();
while (p < q) {
 int j = partition(a, p, q);
 if ((j - p) < (q - j)) {
  stack.push(j + 1);
  stack.push(q);
  q = j - 1;
  } else {
  stack.push(p);
   stack.push(j - 1);
   p = j + 1;
}}}}
```

```
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of elements: ");
int n = sc.nextInt();
int[] a = new int[n];
System.out.println("Enter elements:");
for (int i = 0; i < n; i++)
a[i] = sc.nextInt();
quickSort(a, n);
System.out.println("Sorted array:");
for (int i = 0; i < n; i++)
System.out.print(a[i] + " ");
sc.close();
}
}
```

Input / Output:

```
Enter number of elements: 8
Enter elements:
12 45 76 22 6 98 1 3
Sorted array:
1 3 6 12 22 45 76 98
```

Problem Statement: Implement Merge Sort algorithm to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Time Complexity: O(n logn)

Program:

```
import java.util.*;
public class MergeSortAnalysis {
 static void merge(int arr[], int low, int mid, int high) {
 int i = low, j = mid + 1, k = 0;
 int temp[] = new int[high - low + 1];
 while (i <= mid && j <= high) {
 if (arr[i] <= arr[j])
 temp[k++] = arr[i++];
 else
 temp[k++] = arr[j++];
 }
 while (i <= mid)
 temp[k++] = arr[i++];
 while (j <= high)
 temp[k++] = arr[j++];
 for (i = low, k = 0; i <= high; i++, k++)
 arr[i] = temp[k];
 }
 static void mergeSort(int arr[], int low, int high) {
 if (low < high) {
 int mid = (low + high) / 2;
 mergeSort(arr, low, mid);
 mergeSort(arr, mid + 1, high);
 merge(arr, low, mid, high);
 }}
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 Random rand = new Random();
 System.out.print("Enter number of different n values: ");
 int t = sc.nextInt();
 System.out.println("Enter the n values:");
int n_values[] = new int[t];
 for (int i = 0; i < t; i++) {
 n_values[i] = sc.nextInt();
 }
```

```
    System.out.println("n\tTime (ms)");
    for (int n : n_values) {
    int arr[] = new int[n];
    for (int i = 0; i < n; i++) {
    arr[i] = rand.nextInt(100000);
    }
    long startTime = System.nanoTime();
    mergeSort(arr, 0, n - 1);
    long endTime = System.nanoTime();
    double timeTaken = (endTime - startTime) / 1e6;
    System.out.println(n + "\t" + timeTaken);
    }
    sc.close();
    }
  }
```

Input / Output:

```
Enter number of different n values: 5
Enter the n values:
10 50 100 150 200
n        Time (ms)
10       0.015
50       0.0216
100      0.0673
150      0.1366
200      0.1885
```

Problem Statement: Implement Strassen algorithm for matrix multiplication and analyse its running time.

Time Complexity: O(n2.81)

Program:

```
import java.util.*;
public class MergeSortAnalysis {
 static int[][] add(int A[][], int B[][]) {
 int n = A.length;
 int C[][] = new int[n][n];
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 C[i][j] = A[i][j] + B[i][j];
 return C;
 }
 static int[][] subtract(int A[][], int B[][]) {
 int n = A.length;
 int C[][] = new int[n][n];
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 C[i][j] = A[i][j] - B[i][j];
 return C;
 }
 static int[][] strassen(int A[][], int B[][]) {
 int n = A.length;
 int C[][] = new int[n][n];
 if (n == 1) {
 C[0][0] = A[0][0] * B[0][0];
 return C;
 }
 int size = n / 2;
 int A11[][] = new int[size][size];
 int A12[][] = new int[size][size];
 int A21[][] = new int[size][size];
 int A22[][] = new int[size][size];
 int B11[][] = new int[size][size];
 int B12[][] = new int[size][size];
 int B21[][] = new int[size][size];
 int B22[][] = new int[size][size];
 for (int i = 0; i < size; i++) {
 for (int j = 0; j < size; j++) {
```

```
A11[i][j] = A[i][j];
 A12[i][j] = A[i][j + size];
 A21[i][j] = A[i + size][j];
 A22[i][j] = A[i + size][j + size];
 B11[i][j] = B[i][j];
 B12[i][j] = B[i][j + size];
 B21[i][j] = B[i + size][j];
 B22[i][j] = B[i + size][j + size];
 } }
 int M1[][] = strassen(add(A11, A22), add(B11, B22));
 int M2[][] = strassen(add(A21, A22), B11);
 int M3[][] = strassen(A11, subtract(B12, B22));
 int M4[][] = strassen(A22, subtract(B21, B11));
 int M5[][] = strassen(add(A11, A12), B22);
 int M6[][] = strassen(subtract(A21, A11), add(B11, B12));
 int M7[][] = strassen(subtract(A12, A22), add(B21, B22));
 int C11[][] = add(subtract(add(M1, M4), M5), M7);
 int C12[][] = add(M3, M5);
 int C21[][] = add(M2, M4);
 int C22[][] = add(subtract(add(M1, M3), M2), M6);
 for (int i = 0; i < size; i++) {
 for (int j = 0; j < size; j++) {
 C[i][j] = C11[i][j];
 C[i][j + size] = C12[i][j];
 C[i + size][j] = C21[i][j];
 C[i + size][j + size] = C22[i][j];
 } }
 return C;
 }
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 System.out.print("Enter size of matrix (power of 2): ");
 int n = sc.nextInt();
 int A[][] = new int[n][n];
 int B[][] = new int[n][n];
 System.out.println("Enter elements of Matrix A:");
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 A[i][j] = sc.nextInt();
 System.out.println("Enter elements of Matrix B:");
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
```

```
B[i][j] = sc.nextInt();
 int C[][] = strassen(A, B);
 System.out.println("\nResultant Matrix:");
 for (int i = 0; i < n; i++) {
 for (int j = 0; j < n; j++) {
 System.out.print(C[i][j] + " ");
 }
 System.out.println();
 }
 sc.close();
 }
 }
```

Input / Output:

```
Enter size of matrix (power of 2): 4
Enter elements of Matrix A:
1 2 3 4
4 3 2 1
6 7 8 9
9 8 7 6
Enter elements of Matrix B:
4 5 6 7
7 6 5 4
0 9 8 7
7 8 9 0

Resultant Matrix:
46 76 76 36
44 64 64 54
136 216 216 126
134 204 204 144
```

Problem Statement: Job scheduling Consider the problem of scheduling n jobs of known deadlines d1, d2,...,dn for execution by a single processor. You will earn a profit if a job is completed by its deadline. The jobs can be executed in any order, one job at a time. You want to find a schedule that maximizes the profit. Implement a greedy algorithm for this problem.

Time Complexity: $O(n^2)$

Program:

```
import java.util.*;
class Job {
    int id, deadline, profit;

    Job(int id, int deadline, int profit) {
        this.id = id;
        this.deadline = deadline;
        this.profit = profit;
    } }

public class JobSchedulingInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of jobs: ");
        int n = sc.nextInt();
        Job[] jobs = new Job[n];
        for (int i = 0; i < n; i++) {
             System.out.println("Enter deadline and profit for Job " +
(i + 1) + ":");
            int d = sc.nextInt();
            int p = sc.nextInt();
            jobs[i] = new Job(i + 1, d, p);
        }
        scheduleJobs(jobs);}
    static void scheduleJobs(Job[] jobs) {
        Arrays.sort(jobs, (a, b) -> b.profit - a.profit);
        int n = jobs.length;
        boolean[] slot = new boolean[n];
        int[] result = new int[n];
        int totalProfit = 0;
        for (int i = 0; i < n; i++) {
            for (int j = Math.min(n,jobs[i].deadline)-1;j>=0;j--) {
                if (!slot[j]) {
                    slot[j] = true;
                    result[j] = jobs[i].id;
```

```
totalProfit += jobs[i].profit;
 break; } } }
 System.out.println("\nSelected Jobs:");
 for (int i = 0; i < n; i++) {
 if (slot[i]) {
 System.out.print("Job" + result[i] + " "); }
 }
 System.out.println("\nTotal Profit: " + totalProfit);
 }
}
```

Input / Output:

```
Enter number of jobs: 5
Enter deadline and profit for Job 1:
2 100
Enter deadline and profit for Job 2:
1 19
Enter deadline and profit for Job 3:
2 27
Enter deadline and profit for Job 4:
1 25
Enter deadline and profit for Job 5:
3 15

Selected Jobs:
Job3 Job1 Job5
Total Profit: 142
```

Problem Statement: Implement 0/1 Knapsack problem using Greedy Method.

Time Complexity: O(n log n)

Program:

```
import java.util.*;
class Item {
    int weight, profit;
    Item(int weight, int profit) {
        this.weight = weight;
        this.profit = profit;
    }}
public class KnapsackGreedyInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of items: ");
        int n = sc.nextInt();
        Item[] items = new Item[n];
        for (int i = 0; i < n; i++) {
            System.out.println("Enter weight and profit for item "
+ (i + 1) + ":");
            int w = sc.nextInt();
            int p = sc.nextInt();
            items[i] = new Item(w, p); }
        System.out.print("Enter knapsack capacity: ");
        int capacity = sc.nextInt();
        knapsack(items, capacity); }
    static void knapsack(Item[] items, int capacity) {
        Arrays.sort(items, (a, b) ->
                            Double.compare((double)b.profit/b.weight,
(double)a.profit/ a.weight) );
        int totalProfit = 0;
        System.out.println("\nSelected Items:");
        for (Item item : items) {
            if (capacity >= item.weight) {
                capacity -= item.weight;
                totalProfit += item.profit;
                      System.out.println("Weight: "+item.weight+",
Profit: " + item.profit);
 } }
 System.out.println("Total Profit: " + totalProfit);
 } }
```

Input / Output:

```
Enter number of items: 3
Enter weight and profit for item 1:
10 60
Enter weight and profit for item 2:
20 100
Enter weight and profit for item 3:
3 120
Enter knapsack capacity: 50

Selected Items:
Weight: 3, Profit: 120
Weight: 10, Profit: 60
Weight: 20, Profit: 100
Total Profit: 280
```

Problem Statement: Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Time Complexity: O(e loge + e n)

Program:

```
import java.util.*;
class Kruskal {
 static int find(int[] p, int i) {
 while (p[i] != i)
 i = p[i];
 return i;
 }
 static void union(int[] p, int a, int b) {
 p[a] = b;
 }
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 System.out.println("Enter number of vertices");
 int n = sc.nextInt();
 System.out.println("Enter number of edges");
 int e = sc.nextInt();
 int[][] ed = new int[e][3];
 System.out.println("Enter edges (u v weight)");
 for (int i = 0; i < e; i++) {
 ed[i][0] = sc.nextInt();
 ed[i][1] = sc.nextInt();
 ed[i][2] = sc.nextInt();
 }
 Arrays.sort(ed, (a, b) -> a[2] - b[2]);
 int[] p = new int[n];
 for (int i = 0; i < n; i++)
 p[i] = i;
 int count = 0, cost = 0;
 System.out.println("Spanning tree:");
 for (int i = 0; i < e && count < n - 1; i++) {
 int u = ed[i][0];
 int v = ed[i][1];
 int a = find(p, u);
 int b = find(p, v);
 if (a != b) {
 union(p, a, b);
 System.out.println(u + " - " + v);
 cost += ed[i][2];
```

```
 count++;
 }
 }
 if (count != n - 1)
 System.out.println("No spanning tree");
 else
 System.out.println("Total cost = " + cost);
 }
}
```

Input / Output:

```
Enter number of vertices
5
Enter number of edges
7
Enter edges (u v weight)
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9
Spanning tree:
0 - 1
1 - 2
1 - 4
0 - 3
Total cost = 16
```

Problem Statement: Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Time Complexity: $O(n^2)$

Program:

```
import java.util.*;
import java.io,*;
class Prim {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 System.out.println("Enter number of vertices");
 int n = sc.nextInt();
 int i,j;
 int[][] cost = new int[n][n];
 System.out.println("Enter cost matrix");
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 {
 cost[i][j] = sc.nextInt();
 if (cost[i][j] == 0)
 cost[i][j] = 999;
 }
 int[] visited = new int[n];
 visited[0] = 1;
 int mincost = 0;
 System.out.println("Spanning tree edges:");
 for (int k = 0; k < n - 1; k++)
 {
 int min = 999, u = 0, v = 0;
 for (int i = 0; i < n; i++)
 {
 if (visited[i] == 1) {
 for (int j = 0; j < n; j++)
 {
 if (visited[j] == 0 && cost[i][j] < min)
 {
 min = cost[i][j];
 u = i;
 v = j;
 } } } }
 System.out.println(u + " - " + v);
 visited[v] = 1;
```

```
mincost += min; }
 System.out.println("Total cost = " + mincost);
 }}
```

Input / Output:

```
Enter number of vertices
5
Enter cost matrix
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
Spanning tree edges:
0 - 1
1 - 2
1 - 4
0 - 3
Total cost = 16
```

Problem Statement: Write program to find optimal ordering of matrix multiplication. (Note: Use Dynamic programming method).

Time Complexity: $O(n^2)$

Program:

```
import java.util.*;
public class MatrixChainMultiplication {
 static int[][] m;
 static int[][] s;
 static void matrixChainOrder(int[] p,int n) {
 m=new int[n][n];
 s=new int[n][n];
 for (int i=1;i<n;i++){
 m[i][i]=0; }
 for(int L=2;L<n;L++) {
 for (int i=1;i<n-L+1;i++) {
 int j=i+L-1;
 m[i][j]=Integer.MAX_VALUE;
 for(int k=i;k<j;k++){
 int cost=m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j];
 if(cost<m[i][j]){
 m[i][j]=cost;
 s[i][j]=k; } } } } }
 static void printOptimalParens(int i,int j){
 if(i==j){
 System.out.print("A"+i);
 } else {
 System.out.print("(");
 printOptimalParens(i,s[i][j]);
 printOptimalParens(s[i][j]+1,j);
 System.out.print(")"); } }
 public static void main(String[] args){
 Scanner sc=new Scanner(System.in);
 System.out.print("Enter number of matrices: ");
 int n=sc.nextInt();
 int[] p=new int[n+1];
 System.out.println("Enter dimensions array (size " + (n+1) + "): ");
```

```
(int i=0; i<=n;i++){
 p[i]=sc.nextInt();
 }
 matrixChainOrder(p,n+1);
 System.out.println("\nMinimum number of multiplications: "+m[1][n]);
 System.out.print("Optimal Parenthesization: ");
 printOptimalParens(1,n);
 sc.close();
 }
}
```

Input / Output:

```
Enter number of matrices: 6
Enter dimensions array (size 7):
5 10 3 12 5 50 6

Minimum number of multiplications: 2010
Optimal Parenthesization: ((A1A2)((A3A4)(A5A6)))
```

Problem Statement: Write a program that uses dynamic programming algorithm to solve the optimal binary search tree problem.

Time Complexity: $O(n^3)$

Program:

```
package daa;
import java.util.*;
public class OBST {
    static double[][] c;
    static double[][] w;
    static int[][] r;
    static int find(int i, int j) {
        double min = Double.MAX_VALUE;
        int k = 0;
        for (int m = r[i][j - 1]; m <= r[i + 1][j]; m++) {
            if (c[i][m - 1] + c[m][j] < min) {
                min = c[i][m - 1] + c[m][j];
                k = m;
            } }
        return k;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());
        double[] p = new double[n + 1];
        double[] q = new double[n + 1];
        String[] pInput = sc.nextLine().split(" ");
        for (int i = 1; i <= n; i++)
            p[i] = Double.parseDouble(pInput[i - 1]);
        String[] qInput = sc.nextLine().split(" ");
        for (int i = 0; i <= n; i++)
            q[i] = Double.parseDouble(qInput[i]);
        c = new double[n + 1][n + 1];
        w = new double[n + 1][n + 1];
        r = new int[n + 1][n + 1];
        for (int i = 0; i <= n - 1; i++) {
            w[i][i] = q[i];
            r[i][i] = 0;
            c[i][i] = 0.0;
            w[i][i + 1] = q[i] + q[i + 1] + p[i + 1];
            r[i][i + 1] = i + 1;
            c[i][i + 1] = w[i][i + 1];  }
```

```
w[n][n] = q[n];
r[n][n] = 0;
c[n][n] = 0.0;
for (int m = 2; m <= n; m++) {
for (int i = 0; i <= n - m; i++) {
int j = i + m;
w[i][j] = w[i][j - 1] + p[j] + q[j];
int k = find(i, j);
c[i][j] = w[i][j] + c[i][k - 1] + c[k][j];
r[i][j] = k;
}
}
System.out.println(c[0][n]);
System.out.println(w[0][n]);
System.out.println(r[0][n]);
sc.close();
}
}
```

Input / Output:

```
3
0.15 0.10 0.05
0.05 0.10 0.05 0.05
1.0
0.55
2
```

Problem Statement: From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Time Complexity: $O(n^2)$

Program:

```
import java.util.*;
public class Dijkstra {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int edges = sc.nextInt();
List<List<Node>> adj = new ArrayList<>();
for (int i = 0; i <= n; i++)
adj.add(new ArrayList<>());
for (int i = 0; i < edges; i++) {
int u = sc.nextInt();
int v = sc.nextInt();
double weight = sc.nextDouble();
adj.get(u).add(new Node(v, weight));
}
int source = sc.nextInt();
double[] dist = new double[n + 1];
Arrays.fill(dist, Double.POSITIVE_INFINITY);
dist[source] = 0;
PriorityQueue<Node> pq = new PriorityQueue<>
(Comparator.comparingDouble(node -> node.weight));
pq.add(new Node(source, 0));
while (!pq.isEmpty()) {
Node current = pq.poll();
int u = current.vertex;
if (current.weight > dist[u]) continue;
for (Node neighbor : adj.get(u)) {
                if (dist[u] + neighbor.weight < dist[neighbor.vertex]) {
                    dist[neighbor.vertex] = dist[u] + neighbor.weight;
                    pq.add(new Node(neighbor.vertex,
dist[neighbor.vertex]))}}}
for (int i = 1; i <= n; i++) {
System.out.println("Source " + source + " to " + i + " distance: " +
(dist[i] == Double.POSITIVE_INFINITY ? "INF" : dist[i]));
}
sc.close();}
```

```
static class Node {
int vertex;
double weight;
Node(int vertex, double weight) {
this.vertex = vertex;
this.weight = weight;
}}}
```

Input / Output:

```
3
3
1 2 5
2 3 2
1 3 10
1
Source 1 to 1 distance: 0.0
Source 1 to 2 distance: 5.0
Source 1 to 3 distance: 7.0
```

Problem Statement: Implement N Queen's problem using Back Tracking

Time Complexity: O(n!)

Program:

```
import java.util.*;
public class N_queens {
 static int[] x;
 static int n;
 static int count = 0;
 static boolean place(int k, int i) {
 for (int j = 1; j <= k - 1; j++) {
 if (x[j] == i || Math.abs(x[j] - i) == Math.abs(j - k))
 return false;
 }
 return true;
 }
 static void printBoard() {
 for (int i = 1; i <= n; i++) {
 for (int j = 1; j <= n; j++) {
 if (x[i] == j)
 System.out.print("Q ");
 else
 System.out.print(". ");
 }
 System.out.println();
 }
 System.out.println();
 }
 static void nQueens(int k) {
 for (int i = 1; i <= n; i++) {
 if (place(k, i)) {
 x[k] = i;
 if (k == n) {
 count++;
 System.out.println("Solution " + count + ":");
 printBoard();
 }
 else {
 nQueens(k + 1);
 } } } }
```

```
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of queens: ");
n = sc.nextInt();
x = new int[n + 1];
nQueens(1);
System.out.println("Total solutions: " + count);
sc.close();
}
}
```

Input / Output:

```
Enter number of queens: 4
Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .

Total solutions: 2
```

Problem Statement: Find a subset of a given set S = {sl,s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d. For example, if S= {1, 2, 5, 6, 8} and d = 9 there are two solutions{1,2,6}and{1,8}.A suitable message is to be displayed if the given problem instance doesn't have a solution

Time Complexity: $O(2^n)$

Program:

```
import java.util.*;
public class SumOfSubsets {
 static int[] w;
 static int[] x;
 static int n, target;
 static boolean found = false;
 static void sumOfSub(int s,int k,int r) {
 x[k]=1;
 if(s+w[k]==target){
 found=true;
 printSubset(k);
 }
 else if(k+1<n&&s+w[k]+w[k+1]<=target){
 sumOfSub(s+w[k],k+1,r-w[k]);
 }
 if((s+r-w[k]>=target)&&(k+1<n&&s+w[k+1]<=target)){
 x[k]=0;
 sumOfSub(s,k+1,r-w[k]);
 }
 }
 static void printSubset(int k) {
 System.out.print("{ ");
 for(int i=0;i<=k;i++){
 if(x[i]==1)
 System.out.print(w[i]+" ");
 }
 System.out.println("}");
 }
 public static void main(String[] args) {
 Scanner sc=new Scanner(System.in);
 System.out.print("Enter number of elements: ");
 n=sc.nextInt();
 w=new int[n];
 x=new int[n];
```

```
 System.out.println("Enter elements (sorted in increasing order):");
 for(int i=0;i<n;i++) {
 w[i]=sc.nextInt();
 }
 System.out.print("Enter target sum: ");
 target=sc.nextInt();
 int total=0;
 for(int i=0;i<n;i++){
 total +=w[i];
 }
 sumOfSub(0,0,total);
 if(!found){
 System.out.println("No subset found with given sum.");
 }
 sc.close();
 }
}
```

Input / Output:

```
Enter number of elements: 5
Enter elements (sorted in increasing order):
 1 2 5 6 8
Enter target sum: 9
{ 1 2 6 }
{ 1 8 }
```

Problem Statement: Implement any scheme to find the optimal solution for the Traveling Salesperson problem and then solve the same problem instance using any approximation algorithm and determine the error in the approximation.

Time Complexity: $O(n^2 * 2^n)$

Program:

```
import java.util.*;
public class Tsp {
static final int INF = (int) 1e9;
public static void tsp(int[][] cost, int n) {
int N = 1 << n;
int[][] dp = new int[N][n];
int[][] parent = new int[N][n];
for (int i = 0; i < N; i++) {
Arrays.fill(dp[i], INF);
Arrays.fill(parent[i], -1);
}
dp[1][0] = 0;
for (int mask = 1; mask < N; mask++) {
for (int i = 0; i < n; i++) {
 if ((mask & (1 << i)) != 0) {
 for (int j = 0; j < n; j++) {
 if ((mask & (1 << j)) == 0) {
 int newMask = mask | (1 << j);
 if (dp[mask][i] + cost[i][j] < dp[newMask][j]) {
 dp[newMask][j] = dp[mask][i] + cost[i][j];
 parent[newMask][j] = i;
}}}}}}
int fullMask = (1 << n) - 1;
int lastCity = -1;
int minCost = INF;
for (int i = 1; i < n; i++) {
if (dp[fullMask][i] + cost[i][0] < minCost) {
 minCost = dp[fullMask][i] + cost[i][0];
 lastCity = i;
}}
System.out.println("Minimum Cost: " + minCost);
List<Integer> path = new ArrayList<>();
int mask = fullMask;
int curr = lastCity;
while (curr != -1) {
path.add(curr);
```

```
int prev = parent[mask][curr];
mask = mask ^ (1 << curr);
curr = prev;
}
Collections.reverse(path);
path.add(0);
System.out.print("Path: ");
for (int city : path) {
System.out.print(city + " ");
}}
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[][] cost = new int[n][n];
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
 cost[i][j] = sc.nextInt();
}}
tsp(cost, n);
}}
```

Input / Output:

```
4
0 10 15 20
5 0 9 10
6 13 0 12
8 8 9 0
Minimum Cost: 35
Path: 0 1 3 2 0
```

Problem Statement: Implement All-Pairs Shortest Paths Problem using Floyd's algorithm.

Time Complexity: $O(n^3)$

Program:

```
import java.util.Scanner;
public class Allpaths{
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
double[][] cost = new double[n + 1][n + 1];
double[][] A = new double[n + 1][n + 1];
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= n; j++) {
     cost[i][j] = sc.nextDouble();}}
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= n; j++) {
    A[i][j] = cost[i][j];}}
for (int k = 1; k <= n; k++) {
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        A[i][j] = Math.min(A[i][j], A[i][k] + A[k][j]);}}}
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= n; j++) {
     if (A[i][j] == Double.POSITIVE_INFINITY)
         System.out.print("INF ");
     else
         System.out.print(A[i][j] + " ");}
System.out.println();}
sc.close();}}
```

Input / Output:

```
4
0 3 Infinity 7
8 0 2 Infinity
5 Infinity 0 1
2 Infinity Infinity 0
0.0 3.0 5.0 6.0
5.0 0.0 2.0 3.0
3.0 6.0 0.0 1.0
2.0 5.0 7.0 0.0
```

Problem Statement: Implement Randomized Quick Sort Algorithm (Hint: choose pivot randomly)

Time Complexity: O(n logn)

Program:

```
import java.util.*;
public class rquicksort {
static Random rand = new Random();
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of elements: ");
int n = sc.nextInt();
int[] original = new int[n];
System.out.println("Enter " + n + " elements:");
for(int i=0;i<n;i++)
original[i] = sc.nextInt();
System.out.print("Enter number of runs (different random pivots): ");
int runs = sc.nextInt();
long totalTime = 0;
for(int r=1;r<=runs;r++){
int[] arr = Arrays.copyOf(original,n);
long start = System.nanoTime();
quickSort(arr,0,n-1);
long end = System.nanoTime();
long time = end-start;
totalTime += time;
System.out.println("\nRun " + r);
System.out.println("Sorted Array: " + Arrays.toString(arr));
System.out.println("Execution Time: " + time + " ns");
}
double avg = (double) totalTime / runs;
System.out.println("\nAverage Execution Time: " + avg + " ns");
System.out.println("Estimated Time Complexity: O(n log n)");
sc.close();}
public static void quickSort(int[] arr,int low,int high){
if(low < high){
int pivotIndex = randomizedPartition(arr,low,high);
quickSort(arr,low,pivotIndex-1);
quickSort(arr,pivotIndex+1,high);
}}
private static int randomizedPartition(int[] arr,int low,int high){
int randomIndex = rand.nextInt(high-low+1)+low;
```

```
System.out.println("Random Pivot Index: "+randomIndex+ " Value:
"+arr[randomIndex]);
swap(arr,randomIndex,high);
return partition(arr,low,high);}
private static int partition(int[] arr,int low,int high){
int pivot = arr[high];
int i = low-1;
for(int j=low;j<high;j++){
if(arr[j] <= pivot){
 i++;
 swap(arr,i,j);
}}
swap(arr,i+1,high);
return i+1;
}
private static void swap(int[] arr,int i,int j){
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;
}}
```

Input / Output:

```
Random Pivot Index: 3 Value: 5

Run 1
Sorted Array: [1, 2, 3, 4, 5]
Execution Time: 3599100 ns
Random Pivot Index: 2 Value: 4
Random Pivot Index: 1 Value: 2

Run 2
Sorted Array: [1, 2, 3, 4, 5]
Execution Time: 59500 ns
Random Pivot Index: 3 Value: 5
Random Pivot Index: 0 Value: 3
Random Pivot Index: 0 Value: 1

Run 3
Sorted Array: [1, 2, 3, 4, 5]
Execution Time: 70500 ns
Random Pivot Index: 1 Value: 2
Random Pivot Index: 4 Value: 3
Random Pivot Index: 3 Value: 5

Run 4
Sorted Array: [1, 2, 3, 4, 5]
Execution Time: 65400 ns
Random Pivot Index: 0 Value: 3
Random Pivot Index: 0 Value: 1
Random Pivot Index: 3 Value: 5

Run 5
Sorted Array: [1, 2, 3, 4, 5]
Execution Time: 153400 ns
```

Problem Statement: Print all the nodes reachable from a given starting node in a digraph using BFS method.

Time Complexity: $O(n^2)$

Program:

```
import java.util.*;
public class BFS {
      public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter total number of vertices: ");
            int n = sc.nextInt();
            int[][] adj = new int[n][n];
            System.out.println("Enter the adjacency matrix(row by row)");
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                     adj[i][j] = sc.nextInt();  }}
             System.out.print("Enter the starting node: ");
             int startNode = sc.nextInt();
             System.out.println("\nOrder of nodes from "+startNode+":");
             bfs(adj, startNode, n);
             sc.close();
public static void bfs(int[][] adj, int start, int n) {
       int[] visited = new int[n];
       Queue<Integer> q = new LinkedList<>();
       visited[start] = 1;
       q.add(start);
       while (!q.isEmpty()) {
             int u = q.poll();
             System.out.print(u + " -> ");
             for (int w = 0; w < n; w++) {
                  if (adj[u][w] == 1 && visited[w] == 0) {
                      visited[w] = 1;
                      q.add(w); }}}}}
```

Input / Output:

Input/Output

```
<terminated> Dfs [Java Application] C:\U
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 1
1 0 1 1
1 1 0 1
1 1 1 0
Enter starting vertex: 1
BFS Traversal:
1 0 2 3
```

Problem Statement: Check whether a given graph is connected or not using DFS method.

Time Complexity: $O(n^2)$

Program:

```
package Rquick;
import java.util.*;
public class Dfs {
static void DFS(int[][] graph, int start, int vertices) {
boolean[] visited = new boolean[vertices];
Queue<Integer> queue = new LinkedList<>();
visited[start] = true;
queue.add(start);
while (!queue.isEmpty()){
int node = queue.poll();
System.out.print(node + " ");
for (int i = 0; i < vertices; i++) {
     if (graph[node][i] == 1 && !visited[i]) {
         visited[i] = true;
         queue.add(i);
}}}}
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of vertices: ");
int v = sc.nextInt();
int[][] graph = new int[v][v];
System.out.println("Enter adjacency matrix:");
for (int i = 0; i < v; i++) {
for (int j = 0; j < v; j++) {
     graph[i][j] = sc.nextInt();
}}
System.out.print("Enter starting vertex: ");
int start = sc.nextInt();
System.out.println("DFS Traversal:");
DFS(graph, start, v);}}
```

Input/Output

```
Enter total number of vertices: 3
Enter the adjacency matrix (row by row):

0 1 1
1 0 1
1 1 0
Enter the starting node: 1

Order of nodes traversed (reached) starting from 1:
1 -> 0 -> 2 ->
```